

Bloc 2 MVC PHP TD2

Contexte	2
Exercice 1	2
Exercice 2	3
Exercice 3	6
Exercice 4	7
Exercice 5	9
Exercice 6	10
Exercice 7	11
Exercice 8	12
Exercice 9	14
Conclusion	16

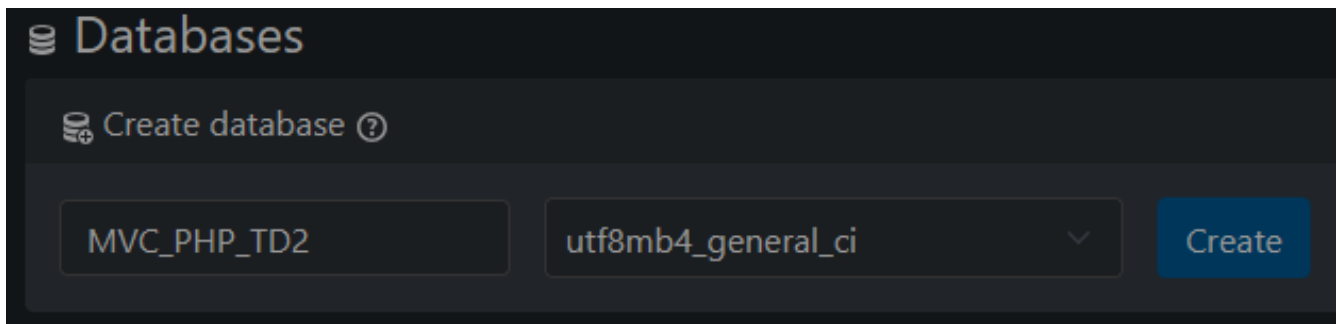
Contexte

Ce TD est la suite du TD1 sur le MVC PHP, il y en aura 4 au total.

(Les fichiers projet seront disponibles dans le portfolio sur **GitHub**).

Exercice 1

Création de la BDD “MVC_PHP_TD2” :



The screenshot shows a 'Databases' interface with a 'Create database' button. Below it, there is a text input field containing 'MVC_PHP_TD2', a dropdown menu showing 'utf8mb4_general_ci', and a blue 'Create' button.

Ajout d’une nouvelle table “voiture” avec 3 champs :

Name	Type ?	Length/Values ?	Default ?	Collation	Attributes	Null	Index
immatriculation Pick from Central Columns	VARCHAR	8	None			☐	PRIMARY
marque Pick from Central Columns	VARCHAR	25	None			☐	---
couleur Pick from Central Columns	VARCHAR	12	None			☐	---

Résultat :

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	immatriculation	varchar(8)	utf8mb4_general_ci		No	None			Change Drop
☐	2	marque	varchar(25)	utf8mb4_general_ci		No	None			Change Drop
☐	3	couleur	varchar(12)	utf8mb4_general_ci		No	None			Change Drop

Ensuite on ajoute des données histoire de :

```
insert into voiture (immatriculation, marque, couleur) values  
( 'AA-000-AA', 'Peugeot', 'Rouge');
```

Exercice 2

Création du fichier “**Conf.php**” dans “C:\xampp\htdocs\public_html\PHP\TD2\”.

```
<?php  
class Conf {  
    static private $databases = array(  
        // ou localhost sur votre machine  
        'hostname' => 'localhost',  
        // A l'IUT, vous avez une BDD nommée comme votre login  
        // Sur votre machine, vous devrez créer une BDD  
        'database' => 'mvc_php_td2',  
        // A l'IUT, c'est votre login  
        // Sur votre machine, vous avez sûrement un compte 'root'  
        'login' => 'root',  
        // A l'IUT, c'est votre mdp (INE par défaut)  
        // Sur votre machine personnelle, vous avez créé ce mdp à l'installation  
        'password' => ''  
    );  
  
    static public function getLogin() {  
        // en PHP l'indice d'un tableau n'est pas forcément un chiffre.  
        return self::$databases['login'];  
    }  
}
```

Le **hostname** est “localhost”, la **BDD** qu'on veut utiliser c'est “mvc_php_td2” et le **login** c'est “root” (il n'y a pas de mot de passe).

Attribut statique : Variable partagée par toutes les instances d'une classe (il n'en existe qu'une seule copie en mémoire).

Méthode statique : Fonction appelée directement via le nom de la classe (**Classe::methode()**) sans avoir besoin de créer d'objet (**new**). Elle n'utilise pas **\$this**.

On crée un fichier **testConf.php** pour vérifier si cela fonctionne, elle va renvoyer “**root**” si elle fonctionne.

```
<?php
    // On inclut les fichiers de classe PHP avec require_once
    // pour éviter qu'ils soient inclus plusieurs fois
    require_once 'Conf.php';
    // On affiche le login de la base de données
    echo Conf::getLogin();
?>
```

On vérifie sur : “http://localhost/public_html/PHP/TD2/testConf.php”

“**root**” s’affiche belle est bien

Ensuite on ajoute le reste dans “**Conf.php**” :

```
static public function getHostname() {
    //en PHP l'indice d'un tableau n'est pas forcément un chiffre.
    return self::$databases['hostname'];
}

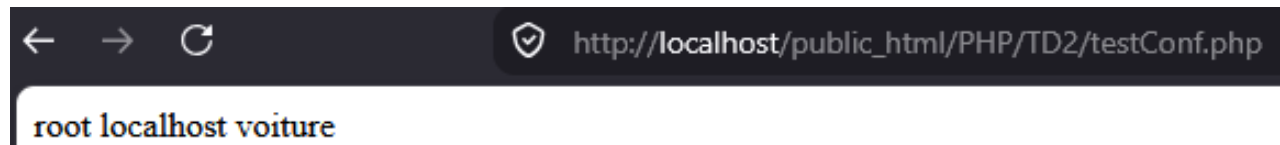
static public function getDatabase() {
    //en PHP l'indice d'un tableau n'est pas forcément un chiffre.
    return self::$databases['database'];
}

static public function getPassword() {
    //en PHP l'indice d'un tableau n'est pas forcément un chiffre.
    return self::$databases['password'];
}
```

On réessaye pour être sur que tout marche dans “*testConf.php*” :

```
<?php
    // On inclut les fichiers de classe PHP avec require_once
    // pour éviter qu'ils soient inclus plusieurs fois
    require_once 'Conf.php';
    // On affiche le login de la base de donnees
    echo Conf::getLogin();
    echo " ";
    echo Conf::getHostname();
    echo " ";
    echo Conf::getDatabase();
    echo " ";
    echo Conf::getPassword();
?>
```

Résultat :

A screenshot of a web browser window. The address bar shows the URL 'http://localhost/public_html/PHP/TD2/testConf.php'. The page content displays the output of the PHP script: 'root localhost voiture'.

← → ↻ http://localhost/public_html/PHP/TD2/testConf.php

root localhost voiture

(Il n’y a pas de mot de passe donc il n’affiche rien).

Exercice 3

On transforme le répertoire en dépôt **Git** :

```
C:\Windows\System32>cd C:\xampp\htdocs\public_html\PHP

C:\xampp\htdocs\public_html\PHP>git init
Initialized empty Git repository in C:/xampp/htdocs/public_html/PHP/.git/

C:\xampp\htdocs\public_html\PHP>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    TD2/

nothing added to commit but untracked files present (use "git add" to track)

C:\xampp\htdocs\public_html\PHP>git add TD2

C:\xampp\htdocs\public_html\PHP>git commit
[master (root-commit) d911fd3] First commit
 2 files changed, 50 insertions(+)
 create mode 100644 TD2/Conf.php
 create mode 100644 TD2/testConf.php

C:\xampp\htdocs\public_html\PHP>git log
commit d911fd31442e1f6c1fc4b3eda0b731da666c330e (HEAD -> master)
Author: RA <aubree.rom@gmail.com>
Date:   Thu Sep 10 14:40:53 2026 +0200

    First commit

C:\xampp\htdocs\public_html\PHP>
```

Il faudra faire “**git add**” et “**git commit**” régulièrement pour sauvegarder nos données.

Exercice 4

Création de “**Model.php**” avec un attribut **public static \$pdo** et une fonction **public static function Init()**.

```
<?php
class Model {
    public static $pdo;

    public static function Init() {
        //
    }
}
?>
```

On relie nos fichiers (**require_once**) :

- On importe le fichier “**Conf.php**” dans notre fichier “**Model.php**”. Cela permet d'avoir accès aux mots de passe et noms d'utilisateurs sans avoir à les réécrire directement ici.

On prépare le “terrain” (les variables) :

- À l'intérieur de la fonction **Init()**, on récupère nos informations de connexion (serveur, **BDD**, identifiant, mot de passe) grâce aux méthodes de la classe **Conf**, et on les stocke dans des variables.

On crée le "pont" vers la **BDD (new PDO)** :

- **PDO** est le traducteur intégré à **PHP** qui permet de parler avec **MySQL**. On lui donne nos variables pour qu'il fabrique la connexion à la **BDD**. On sauvegarde ensuite cette connexion dans **self::\$pdo**.

Lancement automatique (***Model::Init()***):

- En mettant cette ligne à la fin, en dehors de la classe, ça dit à PHP d'exécuter la fonction ***Init()*** immédiatement. Donc dès qu'une page chargera le fichier "**Model.php**", la connexion à la **BDD** sera automatiquement faite.

```
<?php

require_once 'Conf.php';

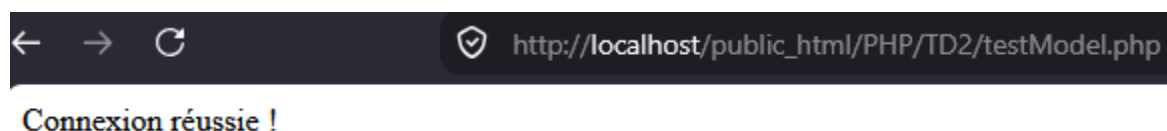
class Model {
    public static $pdo;

    public static function Init() {
        // Identifiant
        $hostname = Conf::getHostname();
        $database_name = Conf::getDatabase();
        $login = Conf::getLogin();
        $password = Conf::getPassword();

        // Connexion PDO
        self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name", $login, $password);
    }
}

// appel fonction pour initialiser
Model::Init();
?>
```

Ensuite, on va créer un "**testModel.php**" pour essayer le code du dessus (comme "**testConf.php**").



The screenshot shows a web browser window with the address bar displaying `http://localhost/public_html/PHP/TD2/testModel.php`. Below the address bar, the text "Connexion réussie !" is displayed, indicating a successful database connection.

La connexion a été effectuée avec succès.

Exercice 5

Gestion des erreurs :

On utilise ici un **try/catch** qui permet d'envoyer un message précis sur la raison d'un crash de manière plus propre et professionnelle.

```
// Connexion PDO
try{
    self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name", $login, $password);
} catch(PDOException $e) {
    echo $e->getMessage(); // affiche un message d'erreur
    die();
}
```

Puis on ajoute ceci :

```
// Connexion PDO
try{
    // Connexion bdd
    self::$pdo = new PDO("mysql:host=$hostname;dbname=$database_name", $login, $password,
        array(PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8")
    );

    // On active le mode d'affichage des erreurs
    self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e) {
    echo $e->getMessage(); // affiche un message d'erreur
    die();
}
```

L'argument **array(PDO::MYSQL_ATTR_INIT_COMMAND => "SET NAMES utf8")** permet de que toutes les chaînes de caractères en entrée et sortie de **MySQL** soient dans le codage **UTF-8**.

La méthode **self::\$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);** active le mode d'affichage des erreurs, ainsi que le lancement d'une exception en cas d'erreur. Grâce à cela, si une requête **SQL** rate par la suite, le script **PHP** générera une erreur claire et s'arrêtera, ce qui est indispensable pour trouver et corriger des bugs rapidement.

Pourquoi il est préférable que la connexion à la **BDD** (stockée dans **Model::\$pdo**) soit un attribut statique ? :

- Car si la connexion n'était pas statique, il faudrait instancier un nouvel objet de la classe **Model** (avec **new Model()**) et donc recréer une nouvelle connexion à la **BDD** à chaque fois que le code en aurait besoin, même sur une seule page. Cela consommerait inutilement des ressources. En rendant cet attribut statique, on garantit qu'il est partagé. La connexion est établie une seule fois et l'unique variable **\$pdo** est disponible partout dans l'application, simplement en appelant **Model::\$pdo**.

Exercice 6

Création du fichier "lireVoiture.php".

Le script charge d'abord le fichier "Model.php" pour récupérer la connexion à la **BDD**. Il exécute ensuite la requête **SQL SELECT * FROM voiture** via la méthode **query** de l'objet **PDO**. Grâce à **fetchAll(PDO::FETCH_OBJ)**, les résultats bruts sont extraits et convertis en un tableau d'objets. Enfin, une boucle **foreach** parcourt ce tableau pour extraire et afficher les informations de chaque voiture.

```
<?php
require_once 'Model.php';

$sql = "SELECT * FROM voiture";
$rep = Model::$pdo->query($sql);

$tab_obj = $rep->fetchAll(PDO::FETCH_OBJ);

echo "<h1>Liste des voitures</h1>";

if (empty($tab_obj)) {
    echo "<p>Aucune voiture trouvée dans la base de données.</p>";
} else {
    foreach ($tab_obj as $obj) {
        echo "<p>";
        echo "Immatriculation : " . $obj->immatriculation . "<br>";
        echo "Marque : " . $obj->marque . "<br>";
        echo "Couleur : " . $obj->couleur;
        echo "</p><hr>";
    }
}
?>
```

Résultat (la taille de “*immatriculation*” dans la **BDD** a été fixé sur **8**, il manque donc une lettre, ça sera fixé dans l'exercice 8) :



Liste des voitures

Immatriculation : AA-000-A
Marque : Peugeot
Couleur : Rouge

Exercice 7

Mise à jour de “**Voiture.php**” (du **TD1**) pour rendre les paramètres du constructeur optionnel :

```
// un constructeur
public function __construct($m = NULL, $c = NULL, $i = NULL) {
    if (!is_null($m) && !is_null($c) && !is_null($i)) {
        $this->marque = $m;
        $this->couleur = $c;
        $this->immatriculation = $i;
    }
}
```

Mise à jour de “**lireVoiture.php**” : pour éviter de gérer l’affichage manuellement, les paramètres du constructeur **Voiture** ont été rendus optionnels et l’inclusion a été pointée vers **TD1**. Ensuite, **PDO::FETCH_CLASS** a été configuré pour que le script convertisse automatiquement chaque ligne **SQL** en un véritable objet métier. Grâce à cela, le programme utilise directement la commande **->afficher()** sur chaque résultat pour centraliser la présentation visuelle de manière très pratique.

Aperçu de “lireVoiture.php” :

```
<?php
require_once 'Model.php';
require_once '../TD1/Voiture.php';

$sql = "SELECT * FROM voiture";
$rep = Model::$pdo->query($sql);

$rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
$tab_voit = $rep->fetchAll();

echo "<h1>Liste des voitures</h1>";

if (empty($tab_voit)) {
    echo "<p>Aucune voiture trouvée.</p>";
} else {
    foreach ($tab_voit as $voit) {
        $voit->afficher();
    }
}
?>
```

Résultat :



Liste des voitures

Voiture AA-000-A de la marque Peugeot (couleur Rouge)

Cela s’affiche de la même manière que dans le **TD1**, a noté aussi que la taille de “*immatriculation*” a été fixée sur **8** (d’où la lettre manquante), ça sera fixé dans l’exercice suivant).

Exercice 8

Le problème actuel est que le script d'affichage contient directement la requête **SQL**, ce qui mélange la mécanique de la **BDD** et l'interface web. Pour régler ce problème, on doit isoler ce code d'extraction pour en faire une méthode statique à l'intérieur de la classe **Voiture**. Ensuite, On doit modifier le fichier **"lireVoiture.php"** pour qu'il utilise directement cette nouvelle fonction. L'objectif est de centraliser la gestion des données au bon endroit.

Mise à jour de **"Voiture.php"** pour y ajouter la fonction **"getAllVoiture()"**

```
public static function getAllVoitures() {
    $sql = "SELECT * FROM voiture";
    // connexion PDO du Model
    $rep = Model::$pdo->query($sql);
    $rep->setFetchMode(PDO::FETCH_CLASS, 'Voiture');
    return $rep->fetchAll();
}

?>
```

Mise à jour de **"lireVoiture.php"** :

```
<?php
require_once 'Model.php';
require_once '../TD1/Voiture.php';

// appel nouvelle methode statique
$tab_voit = Voiture::getAllVoitures();

// affichage
echo "<h1>Liste des voitures</h1>";

if (empty($tab_voit)) {
    echo "<p>Aucune voiture trouvée.</p>";
} else {
    foreach ($tab_voit as $voit) {
        $voit->afficher();
    }
}

?>
```

Exercice 9

Ajout d'une variable debug qui permet de choisir ou non si on veut afficher les messages d'erreurs.

Mise à jour de "**Conf.php**" :

```
// la variable debug est un boolean
static private $debug = True;

static public function getDebug() {
    return self::$debug;
}
}
```

Puis modification du *try/catch* de "**Model.php**" pour adapté l'erreur selon si l'état de "*debug*" :

```
// On active le mode d'affichage des erreurs
self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch(PDOException $e) {
    if (Conf::getDebug()) {
        echo $e->getMessage(); // affiche un message d'erreur technique
    } else {
        echo 'Une erreur est survenue <a href=""> retour a la page d\'accueil </a>';
    }
    die();
}
```

La suite de l'exercice consiste à reproduire la mécanique de l'exercice sur les voitures pour les données de covoiturage, en liant la base de données aux classes **PHP** existantes. (classe *Trajet* et *Utilisateur* du **TD1**)

Création des tables dans la **BDD** :

```
CREATE TABLE utilisateur ( login VARCHAR(32) PRIMARY  
KEY, nom VARCHAR(32), prenom VARCHAR(32) );
```

```
CREATE TABLE trajet ( id INT AUTO_INCREMENT PRIMARY  
KEY, depart VARCHAR(32), arrivee VARCHAR(32), date  
DATE, nbplaces INT, prix INT, conducteur_login  
VARCHAR(32) );
```

Ajout de jeu de données :

```
INSERT INTO utilisateur (login, nom, prenom) VALUES  
( 'jdupont', 'Dupont', 'Jean'),  
( 'mbardet', 'Bardet', 'Marie'),  
( 'elevex', 'Martin', 'Lucas');  
  
INSERT INTO trajet (depart, arrivee, date, nbplaces, prix, conducteur_login) VALUES  
( 'Gap', 'Marseille', '2026-09-15', 3, 20, 'jdupont'),  
( 'Gap', 'Aix-en-Provence', '2026-09-16', 2, 15, 'elevex'),  
( 'Briançon', 'Gap', '2026-09-18', 4, 10, 'mbardet');
```

Modification de “**Utilisateur.php**” pour que son constructeur accepte des arguments vides et une méthode d’extraction globale :

```
<?php  
class Utilisateur {  
    private $login;  
    private $nom;  
    private $prenom;  
  
    public function __construct($l = NULL, $n = NULL, $p = NULL) {  
        if (!is_null($l) && !is_null($n) && !is_null($p)) {  
            $this->login = $l;  
            $this->nom = $n;  
            $this->prenom = $p;  
        }  
    }  
  
    public static function getAllUtilisateurs() {  
        $sql = "SELECT * FROM utilisateur";  
        $rep = Model::$pdo->query($sql);  
        $rep->setFetchMode(PDO::FETCH_CLASS, 'Utilisateur');  
        return $rep->fetchAll();  
    }  
}  
?>
```

Modification de “**Trajet.php**” pour que son constructeur accepte des arguments vides et une méthode d'extraction globale :

```
<?php
class Trajet {
    private $id;
    private $depart;
    private $arrivee;
    private $date;
    private $nbplaces;
    private $prix;
    private $conducteur_login;

    public function __construct($id = NULL, $dep = NULL, $arr = NULL, $dat = NULL, $nb = NULL, $pr = NULL, $cond = NULL) {
        if (!is_null($dep)) {
            $this->id = $id;
            $this->depart = $dep;
            $this->arrivee = $arr;
            $this->date = $dat;
            $this->nbplaces = $nb;
            $this->prix = $pr;
            $this->conducteur_login = $cond;
        }
    }

    public static function getAllTrajets() {
        $sql = "SELECT * FROM trajet";
        $rep = Model::$pdo->query($sql);
        $rep->setFetchMode(PDO::FETCH_CLASS, 'Trajet');
        return $rep->fetchAll();
    }
}
?>
```

Conclusion

Ce TD nous a appris à connecter une application **PHP** à une **BDD MySQL** via l'interface **PDO** afin de rendre nos données persistantes. L'utilisation de **PDO::FETCH_CLASS** nous a montré comment automatiser la transformation des requêtes **SQL** en véritables objets métiers prêts à l'emploi. Enfin, nous avons compris l'importance de structurer notre code en séparant la logique d'accès aux données des scripts d'affichage.